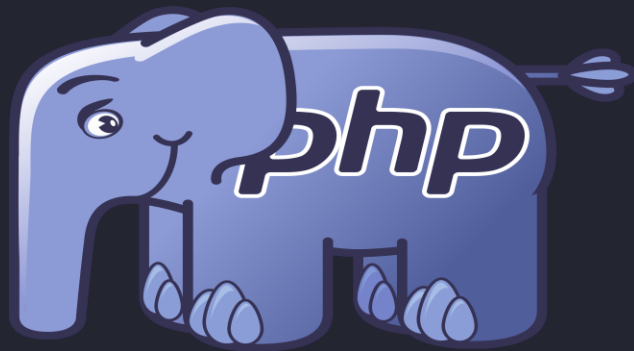




PHP Programming

Fundamentals

Day 01

Prepared by: Mona Ali

Teaching Assistant — Information Technology Institute (ITI)

Agenda

01 History of PHP

02 Why PHP?

03 What do we need? (LAMP Overview)

04 Installing LAMP / XAMPP

05 PHP Overview (Variables, Constants, Flow control, ...)



History of PHP

From personal scripts to a global ecosystem



PHP History Timeline

1994

Rasmus Lerdorf creates PHP as "Personal Home Page" tools

1997

Zeev Suraski & Andi Gutmans rewrite the parser → PHP 3
Renamed to PHP: Hypertext Preprocessor

1999

Zend Engine created → foundation for PHP 4 & 5
Zend Technologies founded

2020+

PHP 8.0 — JIT compilation, union types,
match expressions, named arguments

? WHY PHP?

Easy to Learn

Simple syntax, gentle learning curve

OOP Support

Full object-oriented programming

Cross-Platform

Windows, Linux, Unix, Mac OS X

Server Compatible

Apache, IIS, Nginx and more

Database Support

MySQL, PostgreSQL, SQLite, etc.

Free & Open Source

Download from [php.net](https://www.php.net)



What Do We Need?

- XAMPP — Most popular PHP development environment
- Completely free, easy to install Apache distribution
- Contains MariaDB, PHP, and Perl
- The XAMPP open source package has been set up open source package to be incredibly easy to install and to use

LAMP

Linux + Apache + MySQL + PHP

WAMP

Windows + Apache + MySQL + PHP

MAMP

Mac + Apache + MySQL + PHP



LAMP Stack Overview

LAMP — acronym for a free, open-source software stack

- Linux (Operating System)
- Apache HTTP Server
- MySQL (Database Software)
- PHP / Perl / Python (Programming Language)

Principal components to build a general purpose web server



Installation

- LAMP: bitnami.com/stack/lamp/installer
- XAMPP: apachefriends.org/index.html
- PHP version 7.4+ recommended
- Test by visiting <http://localhost/>



Tip: After installation, create a test.php file with `<?php phpinfo(); ?>` to verify PHP is working correctly.



PHP Overview

Core concepts, syntax, and fundamentals

</> What is a PHP File?

- PHP files can contain text, HTML, JavaScript, and PHP code
- PHP code is executed on the server
- Results returned to the browser as plain HTML
- PHP files have a default extension of ".php"



What PHP Can Do?

- Generate dynamic page content
- Create, open, read, write, and close files on the server
- Collect form data
- Send and receive cookies
- Read, add, delete, modify data in your database
- Restrict users to access some pages
- Encrypt data

</> Embedding PHP in HTML

- Add PHP code inside <?php ... ?> tags within any HTML page
- The PHP interpreter processes the script and replaces it with output

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>PHP</title>
</head>
<body>
  <?php echo "Hello World!"; ?>
</body>
</html>
```

PHP



PHP is Server-Side

PHP is interpreted and executed on the web server, as distinct from JavaScript which runs in the web browser on a user's machine.

A PHP file consists of:

HTML

PHP Tags

PHP Scripts

White Spaces

Comments



PHP Tags

PHP

```
<!-- XML style -->  
<?php echo "<p>Hello!.</p>"; ?>
```

```
<!-- Short style -->  
<? echo "<p>Hello!</p>"; ?>
```

```
<!-- SCRIPT style -->  
<script language="php"> echo "<p>Hello!.</p>"; </script>
```

```
<!-- ASP style -->  
<% echo "<p>Hello!.</p>"; %>
```



PHP Tags — Details

XML style `<?php ?>`

Recommended — portable, can't be disabled by admin

Short style `<? ?>`

Requires `short_open_tag` enabled in config

SCRIPT style

Familiar to JavaScript/VBScript developers

ASP style `<% %>`

Requires `asp_tags` enabled in `php.ini`

>_ PHP Statements & Whitespace

- **echo** — reserved word to display content in browser
- Each statement ends with a semicolon (;)
- Whitespace (spaces, tabs, newlines) is ignored by PHP engine
- Browsers also ignore whitespace in HTML output

```
echo '<p>Hello, World!</p>';
```

```
// Both produce the same result:
```

```
echo 'Hello ';
```

```
echo 'World';
```

```
echo 'Hello ' ; echo 'World'; # but the first version is easier to read
```

PHP

Comments

PHP

```
/* C-style multiline comment:  
    PHP Day01, Instructor: Mona Ali  
    Wish a fruitful Journey with PHP! */  
  
// C++ style single-line comment  
  
# Bash script style single-line comment
```

- ✅ Best Practice: Use comments to document complex logic and make code maintainable.



Go Dynamic — Date Function

The date() function generates dynamic content based on current time

```
<?php
echo "<p>Now, Its ";

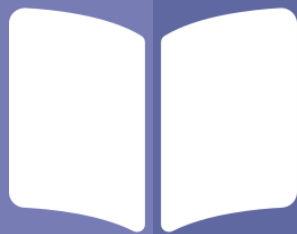
echo date('H:i , jS F Y');

echo "</p>";

?>
```

PHP

Output: Hello, World! Now, Its 17:56 , 22nd March 2021



Variables & Literals

Understanding PHP data storage

“ Variables and Literals

- A value itself is called a literal
- Two kinds of **strings**: double quotation "", and single quotation "".
- PHP evaluates variables inside double-quoted strings: "Hello \$name"
- Single-quoted strings are treated as true literals: 'Hello \$name'
 - Treated as plain text (no variable evaluation).

```
$name = "PHP";
```

```
echo "Hello $name"; // Hello PHP
```

```
echo 'Hello $name'; // Hello $name
```

PHP



Understanding Identifiers

- Identifiers are **names of variables**.
- Rules:
 - Can contain **letters, numbers, underscores** (_).
 - Cannot start with a **digit**.
 - Are **case sensitive**.
- ⚠ A variable can have the same name as a function (not recommended).

PHP Variables — Rules

1

Starts with \$ sign, followed by the name

2

Must begin with a letter or underscore _

3

✓ Can contain only Letters (A-z), Numbers (0-9), and Underscores (_)

4

✗ Should not contain spaces

5

Case-sensitive: \$y and \$Y are different

Declare Your First Variable

- PHP has no command for declaring a variable
- A variable is created the moment you assign a value to it
- **Put quotes around text values**

```
$txt = "Hello world!";  
$x = 5;  
$y = 10.5;
```

```
echo $txt; // Hello world!  
echo $x; // 5
```

PHP

⇔ PHP is a Loosely Typed Language

- No need to tell PHP the **data type of a variable**
- PHP **automatically** converts the variable to the correct data type, depending on its value
- In strongly typed languages, you must declare type and name before use it

PHP (Loosely Typed)

No type declaration required.

```
$x = 5;  
$x = "hello"; // OK!
```

Java (Strongly Typed)

Must define type before using variable.

```
int x = 5;  
String x = "hi"; // Error!
```




Variable Scope

Local, Global, Static, Parameter



Variable Scope

Local

Variables inside a function — accessible only there

Global

Variables outside functions — accessible everywhere except inside functions

Static

Retains value between function calls

Parameter

Passed to function — local to that function

Constants

once declared, are always visible globally;



Local Scope

PHP

```
$x = 5; // global scope
function myTest() {
    $y = 5; // local scope
    echo $y; // Works!
    // echo $x; // Error! $x not accessible
}

myTest();
```



Global Scope

- Variables defined outside any functions have global scope
- can be accessed from any part of the script, **EXCEPT** from within a **function**
- Not accessible inside functions without the **global** keyword

```
$x = 5; // global scope
function myTest() {
    global $x; // Access global $x
    $x = 15;
    var_dump($x); // 15
}

myTest();

var_dump($x); // 15
```

PHP



Static Scope

- When a function is completed, all of its variables are normally deleted. However, sometimes you want a local variable to not be deleted.
- To do this, use the static keyword when you first declare the variable

```
function testStaticFunction(){  
    static $m;  
    $m ++;  
    var_dump($m);  
}  
testStaticFunction(); #1  
testStaticFunction(); #2  
testStaticFunction(); #3
```

PHP



Constants

- Defined using `define()` function or `const` keyword
- No `$` sign when referencing constants
- Constants are globally accessible

```
define("CONSTANT", "Hello world from PHP");
```

```
const TEST = "Welcome";
```

```
echo CONSTANT; // Hello world from PHP
```

```
echo TEST; // Welcome
```

PHP



Parameter Scope

- Parameters are **local** variables whose values are passed to the function by the calling code.
- Parameters are declared in a parameter list as part of the function declaration

PHP

```
function parameterScope($var) {  
    echo $var;  
}  
  
parameterScope(5);           // Output: 5  
  
$name = "Mona";  
parameterScope($name);       // Output: Mona
```

Superglobals

Superglobals are accessible **everywhere** — inside and outside functions.

`$_GET`

Array of variables via GET method

`$_POST`

Array of variables via POST method

`$_REQUEST`

GET + POST + COOKIE combined

`$_COOKIE`

Array of Cookie variables

`$_FILES`

Array of variables related to file uploads

`$_SESSION`

Array of Session variables



Variables Scoping Summary

- Global variables are visible throughout the script, but NOT inside functions
- Use 'global' keyword to access global variables inside functions
- Static variables keep their value between function executions
- Local variables cease to exist when the function terminates
- Superglobals are visible everywhere in a script
- Constants, once declared, are always visible globally



Form Variables

Accessing user input from HTML forms



Form Variables

You may be able to access the contents of the field in the following ways:

- `$field_name` — Short style (requires `register_globals` on, deprecated)
- `$_POST['field']` — Retrieve from POST data
- `$_GET['field']` — Retrieve from GET query string
- `$_REQUEST['field']` — Retrieve from GET, POST, or COOKIE

⚠ Never use `register_globals` in production — it's deprecated and a security risk.



Accessing Form Variables

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    <form action="form.php" method="GET" >
        <input type="text" name="name">

        <input type="password" name="password">

        <input type="submit" >
    </form>
</body>
</html>
```

Form.php

```
var_dump($_GET);
```

```
// Output:
```

```
array (size=2)
    'name' => string 'user@email.com'
    'password' => string 'securePass123'
```



Echo & Print

Output methods in PHP



Echo & Print

echo

Can output multiple strings
No return value
Marginally faster

print

Can output only one string
Returns 1 always
Slightly slower

```
echo "Hello", " world!"; // Works  
echo "Multiple params";
```

```
$var = "PHP Day01";
```

```
print($var); // PHP Day01
```

PHP



Data Types

PHP's type system

PHP Data Types

Integer

Whole numbers

```
$x = 42;
```

String

Text characters

```
$x = "Hello";
```

Array

Multiple values

```
$x = [1,2,3];
```

NULL

No value

```
$x = null;
```

Float

Decimal numbers

```
$x = 3.14;
```

Boolean

True or False

```
$x = true;
```

Object

Class instances

```
$x = new Car();
```

Resource

External reference

```
fopen(...);
```


Variable Casting

- You can temporarily change a variable's type using a cast
- Put the desired type in parentheses before the variable

```
$var1 = 0;  
$var2 = (float)$var1;  
  
// Available casts:  
// (int), (float), (string), (bool), (array), (object)
```

PHP

X¹ Variable Variables

- PHP allows dynamic variable names using \$\$
- You can change the name of a variable dynamically

```
$varname = 'var1';  
  
$$varname = 5; // Same as $var1 = 5;  
  
echo $var1; // 5  
  
echo $$varname; // 5
```

PHP



Operators

Arithmetic, comparison, and more



Arithmetic Operators

Operator	Name	Example
+	Addition	<code>\$x + \$y</code>
-	Subtraction	<code>\$x - \$y</code>
*	Multiplication	<code>\$x * \$y</code>
/	Division	<code>\$x / \$y</code>
%	Modulus	<code>\$x % \$y</code>
**	Exponentiation	<code>\$x ** \$y</code>

```
$result = $a + $b; // Store the result
```

PHP



String Concatenation Operator

- The dot (.) operator joins two strings together
- Works similarly to + for numbers

```
$a = "Hello, ";  
$b = "World!";  
$result = $a . $b;  
  
echo $result; // Hello, World!
```

PHP



Comparison Operators

Operator	Name	Description
<code>==</code>	Equal	true if \$x equals \$y
<code>===</code>	Identical	true if equal AND same type
<code>!=</code>	Not equal	true if \$x not equal \$y
<code><></code>	Not equal	same as !=
<code>></code>	Greater than	true if \$x > \$y
<code><</code>	Less than	true if \$x < \$y
<code><=></code>	Spaceship	returns -1, 0, or 1 (PHP 7+)



Combined Assignment Operators

Assignment	Same as...	Description
<code>x += y</code>	<code>x = x + y</code>	Addition
<code>x -= y</code>	<code>x = x - y</code>	Subtraction
<code>x *= y</code>	<code>x = x * y</code>	Multiplication
<code>x /= y</code>	<code>x = x / y</code>	Division
<code>x %= y</code>	<code>x = x % y</code>	Modulus
<code>\$a .= \$b</code>	<code>\$a = \$a . \$b</code>	Concatenation

Pre/Post Increment & Decrement

- ++ increments, -- decrements a variable by 1
- Pre-increment (++\$a): increments, then returns value
- Post-increment (\$a++): returns value, then increments

```
$a = 4;  
echo ++$a; // Outputs: 5 (value is now 5)  
  
$a = 4;  
echo $a++; // Outputs: 4 (value is now 5)  
  
echo --$a; // Outputs: 4 (decrements back)
```

PHP



Reference Operator (&)

- The & operator creates a reference instead of a copy
- Both \$a and \$b point to the **same piece of memory**. You can change this by unsetting one of them.

```
// Without reference (copy)
$a = 5;
$b = $a;
$a = 7; // $b is still 5
// With reference
$a = 5;
$b = &$a;
$a = 7; // $a AND $b are both 7
```

PHP



Logical operators

Operator	Name	Example
And	True if both \$x and \$y are true	\$x and \$y - \$x && \$y
Or	True if either \$x or \$y is true	\$x or \$y
Not	True if \$x is not true	!\$x
XOr	True if either \$x or \$y is true, but not both	\$x xor \$y

```
$result = $a + $b; // Store the result
```

PHP



Error suppression operator @

- The @ operator is used to suppress error messages.
- The @ operator is used in front of any expression that is, any thing that generates or has a value.
- Without @ → PHP shows a warning
- With @ → Warning is hidden
- ⚠ Not recommended for regular use (use proper error handling instead)

PHP

```
$a = @(25 / 0);  
var_dump($a); // Output: INF  
  
$b = 44 / 0;  
var_dump($b);  
// Warning: Division by zero in on line 77
```



The execution operator ``

- The execution operator uses backticks (`).
- It executes shell commands from within PHP.

PHP

```
$out = `ls -la`;  
echo "<pre>" . $out . "</pre>";  
// Displays the result of the shell command (ls -  
la) in the browser
```



Variable functions

Built in functions



Variable Scope

is_array()

Checks whether the variable is an array

is_double(), is_float(), is_real()

(All the same function): Checks whether the variable is a float

is_long(), is_int(), is_integer()

(All the same function): Checks whether the variable is an integer

is_string()

Checks whether the variable is a string.

Is_bool()

Checks whether the variable is a boolean



Variable Scope

is_object()

Checks whether the variable is an object

Is_resource()

Checks whether the variable is a resource.

Is_null()

Checks whether the variable is null.

Is_numeric()

Checks whether the variable is any kind of number or a numericstring

isset()

Checks if a variable: **exists AND is not NULL** also can pass list of variables `isset($a, $b, $c)`; **unset()** delete variable



Variable Scope

empty()

Checks to see whether a variable exists and has a nonempty, nonzero value; it returns true or false accordingly.

is_callable()

Checks whether the variable is the name of a valid function.

gettype(\$var)

Determines the **data type** of a variable and Returns a **string** representing the type bool, int, double, string, array, object, resource, or NULL.

Settype(\$var,"datatype")

Changes the type of a variable

instanceof ()

is a **type operator** used in **(OOP)** checks whether an object is an instance of a specific class.



Flow Control & Looping

If, switch, for, foreach, while, do-while, break, continue, exit



Conditions

If Condition

```
if (condition) {  
    //code to be executed if this  
    condition is true;  
} elseif (condition) {  
    //code to be executed if first  
    condition is false and this condition is  
    true;  
} else {  
    //code to be executed if all  
    conditions are false;  
}
```

Switch case

```
switch (n) {  
    case label1:  
        // code to be executed if n =  
        label1  
        break;  
    case label2:  
        // code to be executed if n =  
        label2  
        break;  
    case label3:  
        // code to be executed if n =  
        label3  
        break;  
    default:  
        // code to be executed if n does  
        not match any case  
}
```

Loops

For

```
for (expression1; condition; expression2)
{
    // code to be executed
}
```

Foreach

```
foreach ($array as $value) {
    echo $value;
}
```

While

```
while (condition is true) {
    // code to be executed
}
```

Do-while

```
$x = 1000;
do {
    print("welcome to do while looping");
} while ($x < 10);
```

The do...while loop executes the code at least once, then checks the condition and repeat the loop while the specified condition is true.



Break, Continue, Exit

- The **break** statement is used to **terminate the loop completely**.
- The **continue** statement is used to **skip the current iteration** and move to the next one.
- The **exit** statement is used to **stop the entire PHP script execution**.

```
for($i=0;$i<10; $i++){  
    echo "We need the break!";  
    if($i==4) break;  
}  
for($i=0;$i<10; $i++){  
    echo "We need the break! ";  
    if($i==4) continue;  
}  
if (!file_exists("data.txt")) {  
    exit("File not found!");  
}
```

PHP

Add New User

Fill out the information below to create a new workspace account.

First Name

Last Name

Department

Country

Residential Address

Gender

☐ Male
 ☐ Female
 ☐ Other

Username

Password

Professional Skills

☐ HTML
 ☐ CSS
 ☐ JavaScript
 ☐ PHP
 ☐ MySQL
 ☐ Python

Verification

Profile Image

No file chosen

User Details



Alex Rivera

PERSONAL INFORMATION

Department	Country
Open Source	Alex
Gender	Location
Male	123 Indigo Way, Creative City

SKILLS

HTML

CSS

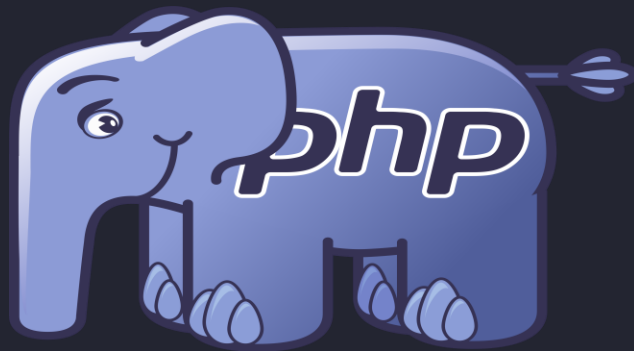
JavaScript

PHP

ACCOUNT CREDENTIALS

Username	arivera_workspace
Password	*****

[Edit Profile](#)



Thank You!

Day 01 Complete — See you in Day 02!

Prepared by Mona Ali — ITI